УЧЕБНИК ПО ЦИКЛУ РАЗРАБОТКИ

От идеи до сопровождения — соло

Этот учебник объединяет знания из шести предыдущих: HTML, CSS, SVG, PHP, SQLite+PHP, Хостинг. Мы рассмотрим полный жизненный цикл веб-сайта глазами одного человека — соло-веб-мастера, который делает всё сам: от сбора требований до обновления и развития.

Аудитория — самостоятельный мастер, уже владеющий основами HTML, CSS, PHP, SQLite и настройки сервера. Учебник не учит технологиям с нуля, а показывает, как их соединить в рабочий процесс. Все этапы — анализ, дизайн, вёрстка, программирование, деплой, запуск, сопровождение, развитие — рассмотрены последовательно, с чек-листами и задачами.

ОГЛАВЛЕНИЕ

- Глава 1. Введение в жизненный цикл сайта
- Глава 2. Окружение разработчика
- Глава 3. Git-workflow для соло-проекта
- Глава 4. Анализ и проектирование
- Глава 5. Вёрстка и дизайн
- Глава 6. Программирование (PHP+БД)
- Глава 7. Деплой на продакшен
- Глава 8. Запуск и аналитика
- Глава 9. Сопровождение и контент-план
- Глава 10. Развитие и итерации
- Глава 11. Безопасность и бэкапы
- Глава 12. Документация проекта
- Глава 13. Тайм-менеджмент, задачи и итоги

ГЛАВА 1. ВВЕДЕНИЕ В ЖИЗНЕННЫЙ ЦИКЛ САЙТА

1.1. Что такое жизненный цикл

Жизненный цикл сайта — это последовательность этапов от возникновения идеи до прекращения поддержки. Сайт не делается один раз и навсегда: он рождается, развивается, иногда умирает. Понимание цикла позволяет планировать работу и не теряться между задачами.

Классические этапы:

- Анализ — что и зачем делаем
- Дизайн — как это выглядит
- Вёрстка — HTML/CSS/SVG по макету
- Программирование — PHP, БД, логика
- Деплой — перенос на боевой сервер
- Запуск — SEO, метрики, анонс
- Сопровождение — контент, бэкапы, мониторинг
- Развитие — новые фичи, рефакторинг
- Завершение — архив, перенос, закрытие

1.2. Особенности соло-разработки

Соло-веб-мастер — это Т-образный специалист: глубокий в одной области (например, вёрстка) и широкий в остальных (сервер, БД, дизайн, копирайтинг, маркетинг). Вы не станете экспертом во всём, но должны понимать достаточно, чтобы сделать работающий сайт от начала до конца.

Главные плюсы соло:

- Полный контроль над решениями
- Никаких совещаний и согласований
- Скорость — от идеи до деплоя за вечер
- Дешевизна — не нужно платить команде

Главные минусы:

- Один источник ошибок — некому проверить
- Узкое горлышко — вы сами, нельзя распараллелить
- Выгорание — трудно переключаться между ролями
- Сложные технологии без команды — предел роста

1.3. Связка с предыдущими учебниками

Этот учебник — кульминация серии. Он не повторяет детально то, что разобрано раньше, а показывает, как соединить шесть дисциплин в один рабочий процесс:

- HTML — структура страниц (глава 5)
- CSS — стили и адаптив (глава 5)
- SVG — иконки и графика (глава 5)
- PHP — бэкенд-логика (глава 6)
- SQLite+PHP — база данных (глава 6)
- Хостинг — деплой и сопровождение (главы 7, 9, 11)

1.4. Типовые ошибки соло-мастера

Опыт собирает шишки. Самые частые ошибки новичков:

- Всё и сразу — пытаться сделать "как у больших"
- Без ТЗ — начать кодить без понимания целей
- Без бэкапов — терять данные при первом сбое
- Без документации — через полгода не вспомнить, что где
- Без Git — ручные бэкапы папок, путаница версий
- Пропустить дизайн — сразу в код, переделывать 5 раз
- Игнорировать безопасность — пока не взломают

1.5. Маленький сайт — что это

В этом учебнике ориентир — малый сайт: лендинг, визитка, корпоративный сайт до 20 страниц. Примеры: сайт-портфолио фотографа, сайт небольшой кофейни, визитка частного юриста, лендинг онлайн-курса.

Малый сайт отличается:

- Бюджет: 5–50 тыс. рублей или бесплатно для себя
- Срок: 1–4 недели от идеи до запуска
- Команда: 1 человек
- Технологии: HTML + CSS + PHP + SQLite
- Хостинг: shared или самый дешёвый VPS ($5/мес)
- Контент: 5–20 страниц, обновляется редко

Для среднего и крупного сайта цикл тот же, но каждый этап длиннее и сложнее. Принципы остаются.

ГЛАВА 2. ОКРУЖЕНИЕ РАЗРАБОТЧИКА

2.1. Что нужно для старта

Прежде чем писать первую строку кода, настройте рабочее место. Минимальный набор соло-мастера:

- Редактор кода (VS Code — бесплатный стандарт)
- Терминал (встроенный в macOS/Linux, Windows Terminal)
- Git (для версионирования)
- PHP локально (для запуска dev-сервера)
- SQLite3 CLI (для работы с БД)
- Браузер с DevTools (Chrome или Firefox)
- SSH-клиент (встроен в ОС)

2.2. VS Code и расширения

VS Code — самый популярный бесплатный редактор. Из коробки подходит для HTML/CSS, но для PHP нужны расширения:

- PHP Intelephense — автодополнение и подсказки
- PHP DocBlocker — генерация комментариев
- PHP Intellisense — навигация по коду
- Live Server — автообновление HTML в браузере
- GitLens — подсказки по Git прямо в коде
- Prettier — автоформатирование
- SQLite Viewer — просмотр .sqlite файлов

Установка через CLI:

```
code --install-extension bmewburn.vscode-intelephense-client
code --install-extension ritwickdey.liveserver
code --install-extension eamodio.gitlens
code --install-extension esbenp.prettier-vscode
code --install-extension qwtel.sqlite-viewer
```

2.3. Локальный сервер PHP

PHP имеет встроенный веб-сервер для разработки. Запустите его в папке проекта:

```
cd ~/projects/mysite
php -S localhost:8000

# Теперь сайт доступен:
# http://localhost:8000
```

Это всё, что нужно для разработки. Никакого Apache, Nginx, Docker, Composer для малого сайта. Откройте URL в браузере — и работает.

2.4. Структура папок проекта

Создайте логичную структуру сразу. Типичная для соло-проекта:

```
mysite/
    public/           # корень сайта
        index.php     # точка входа
        .htaccess     # настройки Apache
        assets/
            css/
            js/
            img/
    app/              # PHP-код (не доступен через веб)
        Database.php
        Models/
        Controllers/
    data/             # НЕ доступен через веб
        database.sqlite
    config/
        db.php
    migrations/
        001_create_users.sql
    README.md
    CHANGELOG.md
    .env               # секреты, в .gitignore!
    .env.example       # шаблон, в Git
    .gitignore
```

Главный принцип: всё, что доступно через веб, — в public/. PHP-код, БД, конфиги — вне public. Это безопасность по умолчанию.

2.5. .env и секреты

Файл .env хранит секреты: пароли, ключи API. Никогда не коммитьте его в Git. Создайте .env.example как шаблон с пустыми значениями:

```
# .env.example
DB_PATH=./data/database.sqlite
SMTP_HOST=
SMTP_USER=
SMTP_PASS=
DEBUG=true

# При установке проекта:
# cp .env.example .env
# nano .env  # заполнить значения
```

2.6. SSH-ключи и доступ к серверу

Если у вас уже есть VPS, настройте SSH-ключи и конфиг. Безопасность и удобство — два плюса сразу:

```
# Создать ключ (один раз)
ssh-keygen -t ed25519 -C "my@machine"

# Скопировать на сервер
ssh-copy-id user@your-server.com

# ~/.ssh/config для удобства
Host mysite
    HostName your-server.com
    User deploy
    IdentityFile ~/.ssh/id_ed25519
    ServerAliveInterval 60

# Теперь: ssh mysite
```

ГЛАВА 3. GIT-WORKFLOW ДЛЯ СОЛО-ПРОЕКТА

3.1. Зачем Git одному

Git — это не только для команд. Даже соло-разработчик получает от него пользу:

- История изменений — что и когда меняли
- Откаты — вернуться к рабочей версии
- Ветки — пробовать фичи, не ломая main
- Бэкап кода на GitHub/GitLab
- Деплой через git pull на сервер
- Issue-трекер для своих же задач

3.2. Инициализация и .gitignore

Создаём репозиторий в папке проекта:

```
cd ~/projects/mysite
git init
git branch -m main  # современный стандарт

# Создаём .gitignore
cat > .gitignore <<EOF
# Секреты
.env

# База данных
data/*.sqlite
data/*.sqlite-journal

# Зависимости (если будут)
vendor/
node_modules/

# Логи и временные файлы
*.log
.DS_Store
Thumbs.db
EOF
```

3.3. Базовый workflow

Простой цикл — для соло-проекта достаточно:

```
# Изменения → индекс → коммит
git add .
git commit -m "feat: добавлена форма обратной связи"

# Запушить на GitHub
git push origin main
```

3.4. Ветки для фич

Хотя вы один, ветки помогают изолировать эксперименты. Если фича затянется или не получится — просто удалите ветку, main останется рабочим.

```
# Создать ветку для фичи
git checkout -b feature/contact-form

# ... работаем ...
git add .
git commit -m "feat: форма обратной связи"

# Готово — вливаем в main
git checkout main
git merge feature/contact-form
git branch -d feature/contact-form  # удалить ветку
```

3.5. Конвенция коммитов

Conventional Commits — стандарт формата сообщений. Помогает понять историю и автоматически генерировать changelog:

- feat: новая функциональность
- fix: исправление бага
- docs: документация
- style: форматирование, без логики
- refactor: переработка без изменения поведения
- test: добавление тестов
- chore: обслуживание проекта, зависимости
- perf: улучшение производительности

```
git commit -m "feat: добавлена корзина"
git commit -m "fix: исправлен расчёт скидки"
git commit -m "docs: обновлён README"
git commit -m "refactor: вынес БД в класс Database"
```

3.6. Теги версий

Когда сайт достигает очередной стабильной версии, пометьте это тегом. Semantic Versioning: MAJOR.MINOR.PATCH.

```
git tag -a v1.0.0 -m "Первый релиз"
git push origin v1.0.0

# MAJOR — несовместимые изменения
# MINOR — новые фичи, обратная совместимость
# PATCH — исправления багов
```

3.7. Откаты

Если что-то сломалось:

```
# Отменить последний коммит,
# изменения останутся в файлах
git reset --soft HEAD~1

# Отменить и стереть изменения
git reset --hard HEAD~1

# Безопаснее — создать revert-коммит
git revert HEAD

# Посмотреть историю
git log --oneline
git log -p path/to/file.php  # с изменениями
```

ГЛАВА 4. АНАЛИЗ И ПРОЕКТИРОВАНИЕ

4.1. Метод 5W

Прежде чем писать код, ответьте на 5 вопросов:

- Who — кто целевая аудитория?
- What — что они должны делать на сайте?
- Where — где они приходят (поиск, соцсети, реклама)?
- When — когда (срок запуска)?
- Why — зачем вам этот сайт?

Пример: сайт-визитка фотографа. Who — невесты 25–35 лет. What — посмотреть портфолио, узнать цены, оставить заявку. Where — Instagram, поиск Яндекса. When — к сезону свадеб (март). Why — получать заказы, не объяснять по телефону.

4.2. Цели сайта

У сайта должна быть измеримая цель. Не «чтобы был», а конкретно:

- Получать 5 заявок в неделю через форму
- Подписать 100 человек на рассылку за месяц
- Продавать 3 курса в месяц
- Привлечь 1000 уникальных посетителей в месяц

Цель определяет структуру сайта. Если цель — заявки, форма должна быть заметной на каждой странице. Если продажи — кнопка «купить» превыше всего.

4.3. Карта сайта

Список всех страниц с иерархией. Для малого сайта это обычно:

```
Главная
    Услуги
        Услуга 1
        Услуга 2
    Портфолио
    О компании
    Блог
        Пост 1
        Пост 2
    Контакты
```

Карта помогает оценить объём работы и спланировать вёрстку. Если страниц больше 20 — это уже не малый сайт, нужна более сложная архитектура.

4.4. Прототип

Прототип — это чёрно-белая раскладка блоков без дизайна. Можно рисовать в Figma, Penpot, или даже на бумаге. Цель — согласовать структуру до того, как вы потратите время на дизайн.

- Где заголовок, где меню, где основной контент
- Какие элементы на каждой странице
- Куда идут ссылки из меню
- Где формы и кнопки

Для соло-мастера прототип — это разговор с самим собой. Нарисовали, посмотрели — что-то не так, переделали за 5 минут. В коде такая переделка заняла бы день.

4.5. Дизайн-макет

Когда прототип устраивает, переходим к макету — цветному, с типографикой, фотографиями, иконками. Инструменты:

- Figma — бесплатная,□□□□
- Penpot — open-source аналог Figma
- Photoshop/GIMP — для тех, кто привык
- На бумаге — для быстрых набросков

Для малого сайта достаточно макета 3–4 ключевых страниц: главная, внутренняя (услуги/блог), контакты. Остальные страницы строятся по тому же шаблону.

4.6. Дизайн-система

Соберите все цвета, шрифты, отступы в один документ. Это сэкономит время на вёрстке:

```
Цвета:
  primary:    #0066cc (синий)
  secondary:  #ff6600 (оранжевый)
  bg:         #ffffff
  text:       #1a1a1a
  muted:      #666666

Шрифты:
  headings: "Montserrat", sans-serif
  body:    "Open Sans", sans-serif

Отступы (8px система):
  xs: 4px   s: 8px   m: 16px
  l: 24px   xl: 32px 2xl: 64px

Размеры шрифтов:
  body: 16px
  h1: 32px   h2: 24px   h3: 20px
  small: 14px
```

ГЛАВА 5. ВЁРСТКА И ДИЗАЙН

5.1. План вёрстки

Когда макеты готовы, переносим их в код. Порядок работ:

- HTML-структура всех страниц
- CSS-стили — общие + компоненты
- Адаптив — mobile-first
- SVG-иконки и графика
- Шрифты через Google Fonts или self-hosted
- Метаданные: title, description, Open Graph

5.2. HTML по стандарту

Используйте HTML5 по учебнику HTML: семантические теги, корректную иерархию заголовков, доступность. Базовый каркас:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
        content="width=device-width,
                 initial-scale=1">
  <title>Мясная лавка «Мясоров»</title>
  <meta name="description"
        content="Свежее мясо с доставкой">
  <link rel="stylesheet"
        href="/assets/css/style.css">
</head>
<body>
  <header>...</header>
  <main>...</main>
  <footer>...</footer>
</body>
</html>
```

5.3. CSS по стандарту

CSS из соответствующего учебника. Mobile-first: сначала мобильные стили, потом медиазапросы для больших экранов. БЭМ или простые классы — на ваш выбор.

```
/* Базовые сбросы */
*, *::before, *::after { box-sizing: border-box; }
body {
  margin: 0;
  font-family: "Open Sans", sans-serif;
  line-height: 1.5;
  color: #1a1a1a;
}

/* Mobile-first */
.container { padding: 0 16px; }

/* Планшет и больше */
@media (min-width: 768px) {
  .container { max-width: 720px;
               margin: 0 auto; }
}

/* Десктоп */
@media (min-width: 1200px) {
  .container { max-width: 1140px; }
}
```

5.4. SVG-графика

Иконки и логотип — в SVG по учебнику. Inline SVG для интерактивных иконок с hover-эффектами, внешний спрайт для набора иконок:

```
<!-- public/assets/icons.svg -->
<svg xmlns="http://www.w3.org/2000/svg"
     style="display:none">
  <symbol id="i-phone" viewBox="0 0 24 24">
    <path fill="currentColor" d="M6 2 L18 22 ..."/>
  </symbol>
</svg>

<!-- в HTML -->
<a href="tel:+78001234567" class="phone">
  <svg width="16" height="16"><use href="/assets/icons.svg#i-phone"/></svg>
  +7 800 123-45-67
</a>
```

5.5. Чек-лист вёрстки

Перед тем как переходить к программированию, проверьте:

- HTML проходит валидацию validator.w3.org
- Все изображения имеют alt
- Адаптив работает на 375px, 768px, 1280px
- Шрифты загружаются с Google Fonts или self-hosted
- Цвета соответствуют дизайн-системе
- Иерархия заголовков: один h1 на странице
- Контакты и формы кликабельны
- Картинки оптимизированы (WebP/AVIF, lazy loading)
- SVG-иконки используют currentColor
- Нет inline-стилей (style=...) — всё в CSS

ГЛАВА 6. ПРОГРАММИРОВАНИЕ (PHP+БД)

6.1. Когда начинается программирование

Если сайт статичный — 5 страниц визитки без форм — PHP не нужен. Просто HTML+CSS. Но как только появляется:

- Форма обратной связи → нужен PHP
- Блог с добавлением постов → нужен PHP + БД
- Корзина в магазине → нужен PHP + БД + сессии
- Авторизация → нужен PHP + БД + сессии

- Поиск по сайту → нужен PHP + БД

6.2. Точка входа index.php

Современный подход — единая точка входа. Все запросы идут через index.php, который маршрутизирует:

```
# public/.htaccess
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]

# public/index.php — точка входа
<?php
require_once __DIR__ . '/../config/db.php';

$uri = parse_url($_SERVER['REQUEST_URI'],
                 PHP_URL_PATH);

switch ($uri) {
    case '/':            require 'home.php'; break;
    case '/contacts':    require 'contacts.php'; break;
    case '/feedback':
        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            // обработка формы
        }
        require 'feedback.php';
        break;
    default:
        http_response_code(404);
        require '404.php';
}
```

6.3. Форма обратной связи

Самая частая задача — форма обратной связи. Связка HTML + PHP по учебникам HTML и PHP:

```
<!-- feedback.php -->
<form method="post">
  <input name="name" required placeholder="Имя">
  <input type="email" name="email"
         required placeholder="Email">
  <textarea name="message" required></textarea>
  <button>Отправить</button>
</form>

<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name = trim($_POST['name']);
    $email = trim($_POST['email']);
    $msg = trim($_POST['message']);

    // Валидация
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        die('Некорректный email');
    }

    // Сохранение в БД
    db()->prepare(
        'INSERT INTO messages
            (name, email, message, created_at)
         VALUES (?, ?, ?, ?)'
    )->execute([$name, $email, $msg, date('c')]);

    // Email уведомление (опционально)
    mail('owner@example.com',
         'Новое сообщение с сайта',
         "От: $name <$email>\n\n$msg");

    header('Location: /feedback?sent=1');
    exit;
}
?>
```

6.4. БД через PDO

Подключение и CRUD — по учебнику SQLite+PHP. Создаём таблицу миграцией:

```
# migrations/001_create_messages.sql
CREATE TABLE messages (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    name        TEXT NOT NULL,
    email       TEXT NOT NULL,
    message     TEXT NOT NULL,
    created_at  TEXT NOT NULL
);

# Применение
sqlite3 data/database.sqlite \
    < migrations/001_create_messages.sql
```

Класс Database как Singleton — из главы 10 учебника по SQLite:

```
# config/db.php
<?php
function db(): PDO {
    static $pdo = null;
    if ($pdo === null) {
        $pdo = new PDO(
            'sqlite:' . __DIR__
                      . '/../data/database.sqlite'
        );
        $pdo->setAttribute(PDO::ATTR_ERRMODE,
                           PDO::ERRMODE_EXCEPTION);
        $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE,
                           PDO::FETCH_ASSOC);
    }
    return $pdo;
}
?>
```

6.5. Чек-лист программирования

- Все запросы к БД — через prepare/execute
- Все пользовательские данные валидируются
- При выводе в HTML — htmlspecialchars()
- Пароли хранятся через password_hash()
- Сессия стартует session_start() в начале скрипта
- CSRF-токены в формах, меняющих состояние
- Ошибки логируются, не показываются пользователю
- Файлы .env и database.sqlite вне public/

ГЛАВА 7. ДЕПЛОЙ НА ПРОДАКШЕН

7.1. Выбор хостинга

Для малого сайта выбор простой. Три варианта:

- Shared (300-800 ₽/мес) — для совсем простых сайтов
- Малый VPS (200-500 ₽/мес) — наш основной выбор
- Бесплатный хостинг — только для теста, не для проды

Shared проще: панель, загрузка файлов по FTP, никаких команд. Но ограничения: нельзя поставить свой пакет, нет root, может быть медленным.

VPS даёт полный контроль и доступность для нашего стека. На нём и работаем по учебнику Хостинг. Российские провайдеры: Timeweb Cloud, Selectel, Rusonyx. Зарубежные: Hetzner, DigitalOcean.

7.2. Домен и DNS

Зарегистрируйте домен у регистратора (Reg.ru, Ru-Center, Namecheap). Запишите A-запись на IP вашего сервера:

```
A  @      203.0.113.10   # mysite.ru
A  www    203.0.113.10   # www.mysite.ru

# MX-записи для почты (если нужна)
MX @ 10   mail.mysite.ru.

# DNS-записи применяются
# от 5 минут до 24 часов
```

7.3. Подготовка сервера

Свежий VPS нужно подготовить. По учебнику Хостинг:

```
# Подключение
ssh root@your-server-ip

# Обновление
apt update &amp;&amp; apt upgrade -y

# Установка стека
apt install -y apache2 php php-sqlite3 \
    php-mbstring php-curl certbot \
    python3-certbot-apache sqlite3

# Создание пользователя (без root)
adduser deploy
usermod -aG sudo deploy

# Настройка SSH-ключей
# (на локальной машине):
ssh-copy-id deploy@your-server-ip
```

7.4. Конфигурация Apache

Создаём VirtualHost:

```
# /etc/apache2/sites-available/mysite.conf
<VirtualHost *:80>
    ServerName mysite.ru
    ServerAlias www.mysite.ru
    DocumentRoot /var/www/mysite/public

    <Directory /var/www/mysite/public>
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog  ${APACHE_LOG_DIR}/mysite_error.log
    CustomLog ${APACHE_LOG_DIR}/mysite_access.log combined
</VirtualHost>

# Включаем и перезапускаем
a2enmod rewrite
a2ensite mysite
systemctl reload apache2
```

7.5. Перенос файлов

Самый простой способ для соло — rsync. Синхронизирует локальную папку public/ с сервером, исключая служебные файлы:

```
rsync -avz --delete \
      --exclude='.git/' \
      --exclude='.env' \
      --exclude='node_modules/' \
      --exclude='*.sqlite' \
      ./public/ \
      deploy@your-server.com:/var/www/mysite/public/

# Не забыть про app/, config/, migrations/
rsync -avz \
      --exclude='.git/' \
      ./app/ ./config/ ./migrations/ \
      deploy@your-server.com:/var/www/mysite/
```

Альтернатива для Git-проектов: клонировать репозиторий прямо на сервере и обновлять через git pull. Это даёт чистую историю версий:

```
# На сервере, один раз
cd /var/www
git clone https://github.com/you/mysite.git

# Обновления
ssh deploy@server
cd /var/www/mysite
git pull origin main
# применить миграции БД
php migrator.php
```

7.6. SSL через Let's Encrypt

По учебнику Хостинг — бесплатный сертификат:

```
certbot --apache -d mysite.ru -d www.mysite.ru

# Спросит:
# - email для уведомлений
# - согласие с условиями
# - перенаправлять ли HTTP→HTTPS (yes)
```

7.7. Права на файлы

Установите правильные права, чтобы Apache мог читать файлы, но не мог менять код (только владелец):

```
# Владелец — deploy, группа — www-data
chown -R deploy:www-data /var/www/mysite

# Каталоги 755, файлы 644
find /var/www/mysite -type d -exec chmod 755 {} \;
find /var/www/mysite -type f -exec chmod 644 {} \;

# Папка data/ — запись для www-data
chmod -R 775 /var/www/mysite/data
chown -R deploy:www-data /var/www/mysite/data

# .env — только владелец
chmod 600 /var/www/mysite/.env
```

7.8. Чек-лист деплоя

- Домен зарегистрирован, DNS настроены
- VPS обновлён, установлен Apache+PHP
- Создан пользователь deploy (не root)
- SSH-ключи настроены, вход по паролю отключён
- Фаервол ufw включен, открыт 80/443/SSH
- VirtualHost создан и включён
- Файлы перенесены через rsync или git pull
- Права 755/644, data/ — 775, .env — 600
- SSL-сертификат получен через certbot
- HTTP→HTTPS редирект работает
- Миграции БД применены на сервере

ГЛАВА 8. ЗАПУСК И АНАЛИТИКА

8.1. Предзапускное тестирование

Перед открытием сайта проверьте:

- Все ссылки работают (нет 404)
- Формы отправляются, письма приходят
- Адаптив на мобильном/планшете/десктопе
- Скорость загрузки < 3 секунд (PageSpeed Insights)
- Валидатор W3C проходит без ошибок
- favicon.ico и apple-touch-icon на месте
- robots.txt и sitemap.xml доступны
- 404 страница информативная

8.2. Метаданные для SEO

Каждая страница должна иметь уникальные:

```
<head>
  <title>Услуга 1 — Компания</title>
  <meta name="description"
        content="Описание услуги, 150-160 символов">
  <meta name="keywords" content="услуга, город">

  <!-- Open Graph для соцсетей -->
  <meta property="og:title" content="Услуга 1">
  <meta property="og:description" content="...">
  <meta property="og:image"
        content="https://mysite.ru/og.jpg">
  <meta property="og:url"
        content="https://mysite.ru/service-1">

  <!-- Twitter Card -->
  <meta name="twitter:card" content="summary_large_image">
  <meta name="twitter:title" content="Услуга 1">
</head>
```

8.3. robots.txt и sitemap.xml

robots.txt управляет индексацией, sitemap.xml помогает поисковикам найти все страницы:

```
# public/robots.txt
User-agent: *
Disallow: /admin/
Disallow: /api/
Allow: /
Sitemap: https://mysite.ru/sitemap.xml

# public/sitemap.xml (генерировать)
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://mysite.ru/</loc>
    <changefreq>weekly</changefreq>
    <priority>1.0</priority>
  </url>
  <url>
    <loc>https://mysite.ru/contacts</loc>
    <priority>0.8</priority>
  </url>
</urlset>
```

8.4. Яндекс.Метрика и Google Analytics

Подключите аналитику. Яндекс.Метрика — для русскоязычной аудитории, Google Analytics — универсально. Код вставляется перед </head>:

```
<!-- Яндекс.Метрика -->
<script type="text/javascript">
   (function(m,e,t,r,i,k,a){
     m[i]=m[i]||function(){(m[i].a=m[i].a||[]).push(arguments)};
     m[i].l=1*new Date();
     k=e.createElement(t), a=e.getElementsByTagName(t)[0]
     k.async=1; k.src=r; a.parentNode.insertBefore(k,a)
   })(window, document, "script",
       "https://mc.yandex.ru/metrika/tag.js", "ym");
   ym(XXXXXXXX, "init", {
       clickmap:true, trackLinks:true,
       accurateTrackBounce:true
   });
</script>
<noscript>
  <div><img src="https://mc.yandex.ru/watch/XXXXXXXX"
            style="position:absolute; left:-9999px;" alt=""/></div>
</noscript>
```

8.5. Яндекс.Вебмастер и Search Console

Зарегистрируйте сайт в Яндекс.Вебмастере и Google Search Console. Добавьте:

- sitemap.xml — для индексации
- Проверка права собственности (HTML-файл в корне)

- Мониторинг ошибок индексации
- Статистика запросов в поиске
- Уведомления о проблемах с сайтом

8.6. Чек-лист запуска

- Сайт работает на https://
- Все страницы отдаются без 404
- В консоли браузера нет ошибок JS
- Метаданные заполнены для каждой страницы
- robots.txt и sitemap.xml доступны
- Сайт добавлен в Яндекс.Вебмастер и Search Console
- Установлена Яндекс.Метрика или GA
- favicon.ico на месте
- 404 страница информативная
- Сайт анонсирован в соцсетях/рассылке

ГЛАВА 9. СОПРОВОЖДЕНИЕ И КОНТЕНТ-ПЛАН

9.1. Что значит «сопровождать»

Запуск — не конец работы. Сайт нужно регулярно обслуживать. Если бросить, через год получите: устаревший PHP с дырами безопасности, переполненную БД, упавший сертификат, сломанные ссылки, провалившийся SEO-трафик.

9.2. Регулярные задачи

Распределите задачи по частоте:

Ежедневно (5 минут):

- Проверить, что сайт открывается
- Посмотреть логи ошибок: tail -50 /var/log/apache2/error.log
- Просмотреть новые сообщения с сайта

Еженедельно (30 минут):

- Бэкап БД и файлов
- Проверить наличие новых ошибок в логах

- Ответить на письма пользователей
- Проверить метрику: трафик, источники, поведение

Ежемесячно (2 часа):

- Установить обновления ОС: apt update && apt upgrade
- Обновить PHP-пакеты (если есть composer)
- Проверить срок SSL-сертификата (certbot renew --dry-run)
- Проанализировать запросы в Яндекс.Вебмастере
- Добавить минимум 1 новую страницу/пост в блог

Раз в полгода:

- Полный аудит безопасности
- Рефакторинг: вычистить «технический долг»
- Пересмотреть дизайн-систему
- Обновить фото и контент

9.3. Скрипт бэкапа

Простейший скрипт для автоматического бэкапа. Поместите в /home/deploy/backup.sh:

```
#!/bin/bash
# /home/deploy/backup.sh
set -e

DATE=$(date +%Y%m%d-%H%M)
BACKUP_DIR=/home/deploy/backups
SITE_DIR=/var/www/mysite

mkdir -p $BACKUP_DIR

# Бэкап БД
sqlite3 $SITE_DIR/data/database.sqlite \
    ".dump" | gzip > \
    $BACKUP_DIR/db-$DATE.sql.gz

# Бэкап файлов
tar -czf $BACKUP_DIR/files-$DATE.tar.gz \
    --exclude='*.sqlite' \
    --exclude='*.log' \
    -C /var/www mysite

# Удалить бэкапы старше 30 дней
find $BACKUP_DIR -mtime +30 -delete

echo "[$DATE] Бэкап завершён" >> $BACKUP_DIR/log.txt
```

Запуск по cron каждый день в 3 часа ночи:

```
# crontab -e
0 3 * * * /home/deploy/backup.sh >> /var/log/backup.log 2>&amp;1
```

9.4. Правило 3-2-1 для бэкапов

Профессиональное правило хранения бэкапов:

- 3 копии данных
- 2 разных носителя (диск + облако)
- 1 копия вне офиса/сервера

В нашем случае: оригинал на сервере, копия в backups/ на том же сервере, и третья копия синхронизируется с облачным хранилищем (Яндекс.Диск, S3):

```
# Дополнение к backup.sh
# Синхронизация с облаком (нужен rclone)
rclone copy $BACKUP_DIR/ yandex:backups/mysite/ \
    --max-age 24h --quiet
```

9.5. Мониторинг uptime

Чтобы узнать о падении сайта раньше, чем пользователи — подключите бесплатный мониторинг. UptimeRobot проверяет сайт каждые 5 минут и шлёт email при падении.

Альтернативы: Pingdom, BetterStack. Для соло — бесплатного тарифа UptimeRobot достаточно (50 мониторов, проверка раз в 5 минут).

9.6. Чек-лист еженедельного обслуживания

- Сайт открывается из браузера
- SSL-сертификат не истёк (проверить в браузере)
- Бэкап сделан и не пустой
- Логи ошибок пустые или понятные
- БД не разрослась (ls -lh data/*.sqlite)
- Место на диске не заканчивается (df -h)
- Меню и формы работают

ГЛАВА 10. РАЗВИТИЕ И ИТЕРАЦИИ

10.1. Что значит «развивать»

Развитие — это не бесконечное добавление фич. Это целенаправленное улучшение сайта в соответствии с целями из главы 4. Если цель — 5 заявок в неделю, а фактически 2, развитие это:

- Улучшение формы (больше конверсия)
- A/B-тест заголовков на главной
- Добавление отзывов для доверия
- Ускорение загрузки (PageSpeed)
- SEO-оптимизация под новые запросы

10.2. Цикл разработки новой фичи

Каждая фича проходит стандартный цикл:

```
# 1. Создать ветку
git checkout -b feature/search

# 2. Реализовать локально
#    (HTML + CSS + PHP + миграции БД)

# 3. Протестировать локально
php -S localhost:8000 -t public/

# 4. Закоммитить
git add .
git commit -m "feat: добавлен поиск по сайту"

# 5. Слить в main
git checkout main
git merge feature/search

# 6. Задеплоить на сервер
git push origin main
ssh deploy@server 'cd /var/www/mysite &amp;&amp; git pull'
ssh deploy@server 'cd /var/www/mysite \
    &amp;&amp; php migrator.php'

# 7. Проверить на проде
# 8. Обновить CHANGELOG.md
```

10.3. Признаки «дурно пахнущего» кода

Когда нужно рефакторить:

- Копипаста — одинаковый код в 3+ местах
- Длинные функции — 100+ строк
- Глубокая вложенность — 4+ уровней if/for
- Магические числа и строки без констант

- Функция делает слишком много вещей
- Имена переменных не говорят ничего (data, x, tmp)
- Дублирование логики между PHP и JS

Правило бойскаута: оставляй код чище, чем нашёл. Если открыл файл править баг — заодно причеши соседние строки, переименуй неясную переменную.

10.4. A/B-тесты

Не угадывайте — тестируйте. A/B-тест показывает пользователям две версии и считает, какая лучше. Для малого сайта без инструментов — вручную:

```
<?php
# Простая A/B-раздача
$variant = mt_rand(0, 1) === 0 ? 'A' : 'B';
setcookie('ab_test', $variant, time() + 86400 * 30);
?>

<!-- Версия A -->
<?php if ($variant === 'A'): ?>
  <h1>Лучшие услуги в городе</h1>
<?php else: ?>
  <h1>Услуги с гарантией качества</h1>
<?php endif; ?>

<?php
# В форме передаём вариант
# и позже анализируем конверсию
?>
```

10.5. Сбор обратной связи

Простейший способ собирать обратную связь — кнопка «Сообщить об ошибке» внизу страницы. Форма с одним полем — что не так. Сообщение падает в БД и шлётся email.

Альтернатива — Яндекс.Вебмастер и Metrika показывают тепловые карты кликов и скролла. Если пользователи пытаются кликнуть на некликабельный элемент — сделайте его кликабельным.

10.6. Чек-лист планирования спринта

Раз в неделю планируйте, что сделать в следующие 7 дней:

- Что важнее всего для цели сайта?
- Сколько часов у меня есть на этой неделе?
- Какая задача даёт максимальный эффект за минимальное время?
- Записать 1–3 задачи в TODO.md

- В конце недели — отметить сделанное, перенести остальное

ГЛАВА 11. БЕЗОПАСНОСТЬ И БЭКАПЫ

11.1. Угрозы для малого сайта

Малый сайт — не цель целенаправленной атаки, но боты сканируют весь интернет 24/7. Самые частые угрозы:

- Подбор пароля к SSH/admin через брутфорс
- Эксплуатация устаревших версий PHP/Apache
- SQL-инъекции через формы поиска
- XSS через комментарии и UGC
- Заливка web-shell через загрузку файлов
- DDoS (редко для малого сайта)

11.2. Регулярные обновления

Главное правило безопасности: обновляйте. Большинство взломов — эксплуатация известных уязвимостей, для которых уже есть патч:

```
# Ежемесячно
apt update &amp;&amp;amp; apt upgrade -y

# Если есть Composer
cd /var/www/mysite
composer update

# Перезапуск после обновления PHP
systemctl restart apache2  # или php8.3-fpm
```

11.3. fail2ban и фаервол

По учебнику Хостинг. fail2ban банит IP-адреса с большим числом неудачных входов. ufw закрывает все порты, кроме нужных:

```
# Установка fail2ban
apt install fail2ban
systemctl enable fail2ban

# Фаервол ufw
ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp   # SSH
ufw allow 80/tcp   # HTTP
ufw allow 443/tcp  # HTTPS
ufw enable
```

11.4. Аудит прав

Периодически проверяйте, что права на файлы не разболтались. Особенно после того, как что-то правили вручную через FTP/SFTP:

```
# Скрипт для проверки
find /var/www/mysite -type d ! -perm 755 -print
find /var/www/mysite -type f ! -perm 644 -print
find /var/www/mysite -name '.env' -exec ls -la {} \;

# Если нашли нарушения — выровнять
find /var/www/mysite -type d -exec chmod 755 {} \;
find /var/www/mysite -type f -exec chmod 644 {} \;
chmod 600 /var/www/mysite/.env
chmod 775 /var/www/mysite/data
```

11.5. Реагирование на инцидент

Если сайт взломали:

- 1. Отключить сервер (если возможно) или хотя бы остановить Apache
- 2. Сохранить логи и дамп БД для разбора
- 3. Восстановить из чистого бэкапа
- 4. Сменить все пароли: SSH, БД, admin-панель, email
- 5. Обновить все пакеты: apt upgrade, composer update
- 6. Найти и закрыть дыру, через которую вошли
- 7. Включить сервер, мониторить логи 24 часа

11.6. Чек-лист безопасности (ежемесячно)

- Система обновлена: apt upgrade выполнено
- fail2ban активен: systemctl status fail2ban
- ufw включен: ufw status verbose
- Вход по паролю SSH отключён

- Пользователь root запрещён (PermitRootLogin no)
- SSL-сертификат не истёк (сайт открывается по https://)
- Файл .env недоступен через веб (откройте https://site/.env)
- Права на файлы: 755/644/600/775 как в чек-листе деплоя
- Бэкапы делаются и хотя бы один раз восстанавливались из бэкапа

ГЛАВА 12. ДОКУМЕНТАЦИЯ ПРОЕКТА

12.1. Зачем документировать соло-проект

Через 3 месяца вы не вспомните, почему назвали функцию именно так, что хранит таблица xyz, какой URL у админки. Через полгода — тем более. Документация — это записка самому себе в будущем.

Минимальный набор:

- README.md — что это и как запустить
- CHANGELOG.md — история версий
- TODO.md — список задач
- .env.example — шаблон переменных окружения
- Комментарии в коде — только в сложных местах

12.2. README.md

Главный файл проекта. Должен отвечать на вопросы: что это, как установить, как запустить.

````
# Мясная лавка «Мясоров»

Корпоративный сайт мясной лавки.
Стек: HTML + CSS + SVG + PHP 8.3 + SQLite.

## Установка для разработки

```bash
git clone https://github.com/you/myasorov.git
cd myasorov
cp .env.example .env  # заполнить секреты
sqlite3 data/database.sqlite < migrations/001_init.sql
php -S localhost:8000 -t public/
```

## Деплой на продакшен

См. DEPLOY.md.

## Структура проекта

```
public/      — корень сайта (доступен через веб)
app/         — PHP-код (модели, контроллеры)
config/      — конфиги
data/        — БД и пользовательские файлы
migrations/  — SQL-миграции схемы
```

## Лицензия

MIT. Свободно используйте.
````

12.3. CHANGELOG.md

История изменений. Соответствует тегам в Git:

```
# Changelog

## [1.2.0] — 2026-10-26
### Added
- Форма обратной связи на странице контактов
- Иконки в меню навигации

### Fixed
- Исправлена ошибка в расчёте скидки

## [1.1.0] — 2026-10-15
### Added
- Страница «Услуги» с ценами
- Адаптив для планшетов

## [1.0.0] — 2026-10-01
### Added
- Первый релиз сайта
- Главная, Контакты, О компании
```

12.4. TODO.md

Список задач. Простой и наглядный:

```
# TODO

## Сейчас
- [ ] Добавить страницу «Доставка»
- [ ] Оптимизировать картинки → WebP

## Бэклог
- [ ] Поиск по сайту
- [ ] Корзина для интернет-магазина
- [ ] Админ-панель для блога

## Идеи
- [ ] Интеграция с Telegram для уведомлений
- [ ] Онлайн-запись на консультацию
```

12.5. .env.example

Шаблон переменных окружения. В Git, в отличие от .env. Когда кто-то (или вы через год) захочет поднять проект, он скопирует .env.example в .env и заполнит:

```
# .env.example
# Скопируйте в .env и заполните

DB_PATH=./data/database.sqlite

# SMTP для отправки почты
SMTP_HOST=
SMTP_PORT=587
SMTP_USER=
SMTP_PASS=
MAIL_FROM=noreply@mysite.ru

# Режим отладки (true на dev, false на prod)
DEBUG=true

# Яндекс.Метрика (номер счётчика)
YM_ID=
```

12.6. Комментарии в коде

Не комментируйте очевидное (i++ // увеличиваем i). Комментируйте:

- Почему так, а не иначе (если неочевидно)
- Сложные алгоритмы и формулы
- Костыли и обходные пути (с пометкой HACK)
- TODO с номером задачи: // TODO: оптимизировать #12

```
// Плохо:
$i++; // увеличиваем i

// Хорошо:
// Используем bcadd, потому что обычный +
// теряет точность на больших суммах
$total = bcadd($total, $item_price, 2);

// TODO: вынести в класс Money (#15)
// HACK: обход бага PHP #12345, убрать после обновления до 8.4
```

ГЛАВА 13. ТАЙМ-МЕНЕДЖМЕНТ, ЗАДАЧИ И ИТОГИ

13.1. Метод Помодоро

Техника управления временем: 25 минут работы + 5 минут перерыв. После 4 помодоро — большой перерыв 15-30 минут. Меняете задачи в перерывах, не в pomodoro.

Преимущества:

- Чёткие рамки — мозг сфокусирован
- Регулярные перерывы — меньше выгорание
- Легко оценить время: "это 2 помодоро"
- Внешняя дисциплина: таймер на телефоне

Инструменты: TomatoTimer (веб), Forest (iOS/Android), Pomotodo. Или просто терминальный таймер:

```
# Простой таймер на 25 минут
sleep 1500 &amp;&amp; echo 'Pomodoro done!' &amp;&amp; \
  notify-send 'Pomodoro' 'Перерыв!'
```

13.2. Оценка времени по PERT

Чтобы не обещать себе невыполнимого, оценивайте задачи по трём сценариям:

- O — оптимистичная (всё идёт идеально)
- M — наиболее вероятная
- P — пессимистичная (что-то пойдёт не так)

Формула PERT: (O + 4M + P) / 6. Например, задача «сделать форму обратной связи»: O=2ч, M=4ч, P=10ч. Оценка: (2 + 16 + 10) / 6 ≈ 4.7 часа.

13.3. Правило 80/20

Принцип Парето: 20% усилий дают 80% результата. Для веб-разработки это значит:

- 20% страниц дают 80% трафика (главная, услуги, контакты)
- 20% функций дают 80% ценности (форма, прайс, портфолио)
- 20% времени уходит на 80% функционала (донесение до запуска)
- 80% времени — на полировку оставшихся 20%

Соло-мастеру важно не впасть в перфекционизм. Запустите "минимально жизнеспособный сайт" (MVP), получите первых пользователей, потом дорабатывайте по обратной связи.

13.4. Недельная ретроспектива

Раз в неделю — 30 минут на ретроспективу. Спросите себя:

- Что получилось хорошо на этой неделе?
- Что не получилось и почему?
- Что буду делать иначе на следующей неделе?
- Что нового узнал, чему научился?

Запишите ответы в RETRO.md или просто в блокнот. Через полгода увидите паттерны и свой рост.

13.5. Минимизация выгорания

Соло-разработка — марафон, не спринт. Признаки выгорания:

- Не получается заставить себя открыть редактор
- Лёгкие задачи кажутся неподъёмными
- Появляется раздражение от проекта
- Снижается качество кода, больше багов
- Пропадают идеи и энтузиазм

Профилактика:

- Не работать вечерами и выходными (если не горит)
- Брать 1-2 выходных от проекта в неделю
- Чередовать типы задач: дизайн, код, контент, общение
- Делать физическую разминку каждый час
- Иметь хобби вне веб-разработки
- Общаться с другими: митапы, форумы, чаты

13.6. Задача 1: ТЗ на сайт-визитку

Напишите ТЗ на сайт-визитку по схеме 5W для себя или вымышленного заказчика. Решение:

```
## ТЗ: Сайт-визитка дизайнера Анны

Who: малый и средний бизнес 25-50 лет,
     ищет дизайнера интерьера.
What: посмотреть портфолио, узнать цены,
      оставить заявку на консультацию.
Where: Instagram (60%), поиск Яндекса,
       рекомендации.
When: запуск к 1 марта (к сезону).
Why: получать 3-5 заявок в неделю,
     не тратить время на объяснения.

Цель: 5 заявок через форму в неделю.

Карта сайта (8 страниц):
  Главная
  Портфолио
    Проект 1
    Проект 2
    ...
  Услуги
  Цены
  О себе
  Контакты

Стек: HTML5 + CSS3 + SVG + PHP 8.3 + SQLite.
```

13.7. Задача 2: раскладка по этапам

Разложите сайт-визитку на этапы с оценкой времени по PERT. Решение:

```
Этап                 O    M    P    PERT
──────────────────────────────────────────
Анализ и ТЗ          2ч   4ч   8ч   4.3ч
Прототип в Figma     3ч   6ч   12ч  6.5ч
Дизайн-макеты        4ч   8ч   20ч  9.3ч
Вёрстка HTML/CSS     6ч   10ч  24ч  11.7ч
SVG-иконки          1ч   2ч   4ч   2.2ч
PHP: форма, БД       4ч   8ч   16ч  8.7ч
Контент              3ч   5ч   10ч  5.5ч
Деплой и SSL         1ч   2ч   6ч   2.5ч
Тестирование         1ч   2ч   4ч   2.2чп──────────────────────────────────────────
Итого               25ч  47ч  100ч 53.9ч

Реалистично: 50-55 часов работы.
При 4 часах в день — 13-14 дней.
```

13.8. Задача 3: скрипт бэкапа

Напишите bash-скрипт для ежедневного бэкапа сайта (файлы + БД) с удалением старых. Решение:

```
#!/bin/bash
# /home/deploy/backup.sh
set -e

DATE=$(date +%Y%m%d-%H%M)
BACKUP_DIR=/home/deploy/backups
SITE_DIR=/var/www/mysite
RETENTION_DAYS=30

mkdir -p $BACKUP_DIR

# БД
sqlite3 $SITE_DIR/data/database.sqlite \
    ".dump" | gzip > \
    $BACKUP_DIR/db-$DATE.sql.gz

# Файлы (без БД и логов)
tar -czf $BACKUP_DIR/files-$DATE.tar.gz \
    --exclude='*.sqlite' \
    --exclude='*.sqlite-journal' \
    --exclude='*.log' \
    -C /var/www mysite

# Удаление старых
find $BACKUP_DIR -mtime +$RETENTION_DAYS -delete

# Лог
echo "[$DATE] OK" >> $BACKUP_DIR/backup.log

# Cron: каждый день в 3:00
# 0 3 * * * /home/deploy/backup.sh
```

13.9. Задача 4: регистрация домена + SSL

Опишите по шагам, как зарегистрировать домен mysite.ru и подключить SSL.
Решение:

```
# Шаг 1. Регистрация домена
# Зайти на reg.ru или nic.ru
# Найти свободный домен mysite.ru
# Зарегистрировать (500-2000 ₽/год)

# Шаг 2. Настройка DNS
# В панели регистратора:
# A  @    203.0.113.10
# A  www  203.0.113.10

# Шаг 3. Настройка Apache
# См. главу 7 — VirtualHost с ServerName mysite.ru
# a2ensite mysite &amp;amp;&amp;amp; systemctl reload apache2

# Шаг 4. Получение SSL
certbot --apache -d mysite.ru -d www.mysite.ru
# Согласиться, выбрать redirect HTTP→HTTPS

# Шаг 5. Проверка
# Открыть https://mysite.ru — значок замка
# certbot renew --dry-run  — автообновление работает

# DNS применяются 5 минут - 24 часа.
```

13.10. Задача 5: README.md

Напишите README.md для сайта-визитки. Решение:

````
# Сайт-визитка дизайнера Анны

Сайт-портфолио дизайнера интерьера.
Стек: HTML5, CSS3, SVG, PHP 8.3, SQLite.

## Установка для разработки

```bash
git clone git@github.com:anna/site.git
cd site
cp .env.example .env  # заполнить секреты
sqlite3 data/database.sqlite \
    < migrations/001_init.sql
php -S localhost:8000 -t public/
```

Открыть http://localhost:8000.

## Структура

- `public/` — корень сайта (доступен через веб)
- `app/`    — PHP-код
- `data/`   — SQLite БД
- `migrations/` — SQL-миграции

## Команды

- `php -S localhost:8000 -t public/` — dev-сервер
- `php migrator.php` — применить миграции БД
- `./deploy.sh` — деплой на продакшен

## Лицензия

MIT.
````

13.11. Задача 6: проведение ретроспективы

Проведите ретроспективу прошедшей недели по шаблону. Решение:

```
## Ретроспектива за неделю 20-26 октября 2026

### Что получилось хорошо
- Запустил сайт-визитку Анны в срок
- Получил первый фидбек: 3 заявки
- Настроил автоматический бэкап по cron
- Выучил метод PERT для оценки времени

### Что не получилось
- Не успел сделать страницу "Услуги"
  (оценил в 2ч, заняло 6ч)
- Два вечера работал до 2 ночи, выгорел
- Забыл обновить CHANGELOG.md после релиза

### Что буду делать иначе
- Оценивать по PERT, не на глаз
- Строго заканчивать в 22:00
- После каждого деплоя — коммит в CHANGELOG

### Чему научился
- rsync с --exclude экономит много времени
- fail2ban решил 90% проблем с подбором пароля
- SVG-спрайты через <symbol> удобнее <img>
```

13.12. Задача 7: план спринта

Спланируйте спринт на следующую неделю (4 часа в день, 28 часов всего).
Решение:

```
## Спринт: 27 октября — 2 ноября
Цель: добавить страницу "Услуги"
      + настроить Яндекс.Метрику.

Понедельник (4ч):
  [1ч]  Прототип страницы в Figma
  [2ч]  Вёрстка HTML/CSS (mobile-first)
  [1ч]  Контент: текст + 3 фото услуг

Вторник (4ч):
  [1ч]  SVG-иконки для услуг
  [2ч]  PHP: добавление услуг через БД
  [1ч]  Тестирование локально

Среда (4ч):
  [2ч]  Подключение Яндекс.Метрики
  [1ч]  Настройка целей (клик по кнопке,
        отправка формы)
  [1ч]  Проверка через Tag Assistant

Четверг (4ч):
  [1ч]  Деплой на прод: rsync + миграции
  [1ч]  Регистрация в Яндекс.Вебмастере
  [2ч]  Правки по результатам тестирования

Пятница (4ч):
  [1ч]  Бэкап перед финальным деплоем
  [1ч]  Финальный деплой
  [1ч]  Обновление CHANGELOG, README
  [1ч]  Ретроспектива

Суббота-воскресенье: ВЫХОДНЫЕ
(если всё по плану — не трогать проект)
```

13.13. Итоги по главам

Глава 1. Жизненный цикл сайта — последовательность этапов от идеи до закрытия. Соло-мастер — Т-образный специалист, который делает всё сам, опираясь на знания из шести предыдущих учебников.

Глава 2. Окружение: VS Code с расширениями, локальный сервер php -S, структура папок с public/ и data/, .env для секретов, SSH-ключи для доступа к серверу.

Глава 3. Git-workflow: ветки main + feature/, конвенция коммитов (feat:/fix:/docs:), теги версий, откаты через revert/reset. .gitignore исключает .env и БД.

Глава 4. Анализ: метод 5W (кто/что/где/когда/зачем), измеримые цели, карта сайта, прототип, дизайн-макет, дизайн-система с цветами/шрифтами/отступами.

Глава 5. Вёрстка по учебникам HTML, CSS, SVG: семантические теги, mobile-first, SVG-иконки с currentColor, доступность. Чек-лист валидации и адаптива.

Глава 6. Программирование: точка входа index.php с роутингом, форма обратной связи с валидацией, PDO для SQLite, миграции. Чек-лист безопасности кода.

Глава 7. Деплой: выбор VPS, домен + DNS, подготовка сервера (Apache+PHP+certbot), перенос через rsync/git, SSL через Let's Encrypt, права 755/644/600/775.

Глава 8. Запуск: тесты, метаданные (title, description, Open Graph), robots.txt и sitemap.xml, Яндекс.Метрика и Вебмастер, чек-лист запуска.

Глава 9. Сопровождение: регулярные задачи (день/неделя/месяц/полгода), скрипт бэкапа по cron, правило 3-2-1, мониторинг через UptimeRobot.

Глава 10. Развитие: цикл новой фичи (ветка → код → merge → деплой → проверка), рефакторинг по запахам кода, A/B-тесты, сбор обратной связи, чек-лист спринта.

Глава 11. Безопасность: обновления, fail2ban и ufw, аудит прав, план реагирования на инцидент. Чек-лист ежемесячной проверки.

Глава 12. Документация: README.md, CHANGELOG.md, TODO.md, .env.example, комментарии в коде только где нужно объяснить почему, а не что.

Глава 13. Тайм-менеджмент: метод Помодоро (25/5), оценка PERT (O+4M+P)/6, правило 80/20, недельная ретроспектива, минимизация выгорания. Семь практических задач.

13.14. Что читать дальше

После освоения этого учебника логично двигаться в трёх направлениях. Во-первых, фреймворки — Laravel или Slim. Они решают рутину (роутинг, ORM, миграции, валидация) и дают больше времени на бизнес-логику.

Во-вторых, DevOps и автоматизация: Docker для изоляции окружения, Ansible для настройки серверов, GitHub Actions для CI/CD. Когда сайтов станет 5+ — ручной деплой станет узким горлышком.

В-третьих, бизнес-навыки: оценка стоимости проектов, ведение переговоров с заказчиками, контракты, юридические аспекты. Соло-мастер — это бизнес, не только программирование.

Главный совет: не пытайтесь внедрить сразу все практики из этого учебника. Начните с минимума — Git, базовая структура папок, README, бэкапы. Затем добавляйте по одной привычке в месяц: чек-листы, Помодоро, ретроспективы,

A/B-тесты. Через год у вас будет рабочий процесс, соответствующий вашему стилю и проектам. Опыт складывается из десятков сделанных и переделанных сайтов, а не из прочитанных справочников.
